

Claude Multi-Agent Project Template

A production-ready, research-backed template for building software with a team of specialized Claude Code sub-agents. Optimized for parallel execution, context window management, and high-quality output.

Overview

This template provides a standardized structure for developing software projects using Claude Code's multi-agent capabilities. It implements best practices derived from extensive research into AI agent orchestration, context engineering, and parallel execution patterns.

Key Features

- **12 Specialized Agents:** Pre-configured agents for orchestration, architecture, development, review, testing, and documentation
- **Optimized Context Management:** Progressive disclosure and just-in-time loading keep initial context under 20% of the 200k token limit
- **Three-Phase Workflow:** Parallel planning, sequential implementation, and iterative review
- **Bottom-Up Development:** Supports foundational layer testing before moving up the stack
- **Multi-Level Validation:** Three-tier code review (peer, senior, security) and three-tier testing (unit, integration, system)
- **Reusable Structure:** Copy and customize for any software project

Quick Start

1. **Copy the template** to your project directory
2. **Edit** `claude.md` to describe your project's core objective
3. **Fill in** `plan.md` with your high-level features and requirements
4. **Start building** by invoking agents for planning, implementation, and review

See [USAGE_GUIDE.md](#) for detailed instructions.

Project Structure

Plain Text

```

claude_project_template/
├── claude.md                # Root context (auto-loaded, <60 lines)
├── plan.md                  # Project requirements and task tracking
├── USAGE_GUIDE.md           # Comprehensive usage instructions
├── .claude/
│   └── agents/              # 12 specialized sub-agent definitions
│       ├── orchestrator.md  # NEW: Automated workflow coordination
│       ├── architect.md
│       ├── product_manager.md
│       ├── ux_designer.md
│       ├── ui_developer.md
│       ├── backend_developer.md
│       ├── code_reviewer_I.md
│       ├── code_reviewer_II.md
│       ├── cyber_code_reviewer.md
│       ├── unit_test_agent.md
│       ├── integration_test_agent.md
│       ├── system_test_agent.md
│       └── documentation_specialist.md
├── .gitignore               # Multi-language git ignore patterns
├── .pre-commit-config.yaml  # Automated linting and formatting
├── .env.example             # Environment variables template
└── docs/                   # Detailed docs (loaded on-demand)
    ├── architecture.md
    ├── testing_strategy.md
    ├── code_conventions.md
    └── deployment.md

```

Agent Team

Orchestration

- **Orchestrator:** Automated workflow coordination, agent invocation, feedback loop management

Planning & Design

- **Architect:** System design, technology choices, ADRs
- **Product Manager:** Feature requirements, user stories, task.md generation
- **UX Designer:** User experience, wireframes, design systems

Development

- **UI Developer:** Frontend implementation, responsive design

- **Backend Developer:** APIs, business logic, database interactions

Quality Assurance

- **Code Reviewer I:** Peer review for style, conventions, readability
- **Code Reviewer II:** Senior review for architecture, performance, integration
- **CyberCodeReview:** Security-focused vulnerability analysis
- **Unit Test Agent:** Component-level testing
- **Integration Test Agent:** Component interaction testing
- **System Test Agent:** End-to-end validation

Documentation

- **Documentation Specialist:** API docs, technical guides, user manuals

Workflow

Phase 1: Planning (Parallel Execution)

Bash

```
@product_manager @ux_designer Create task.md for [feature name]
```

- Product Manager and UX Designer work in parallel
- Each has dedicated 200k context window
- Output: Comprehensive `task.md` with requirements and specifications

Phase 2: Implementation (Sequential Handoff)

Bash

```
@backend_developer Implement feature from tasks/feature.md
```

- Developer receives concise task.md
- Uses full 200k context for implementation
- Writes code and unit tests

Phase 3: Review (Iterative Loop)

Bash

```
@code_reviewer_I Review code for [feature]
@code_reviewer_II Review code for [feature]
@cyber_code_reviewer Review code for [feature]
```

- Three-tier review process
- Feedback looped back to developer until all checks pass
- Ensures quality, architecture alignment, and security

Phase 4: Testing (Bottom-Up Validation)

Bash

```
@unit_test_agent Test [feature]
@integration_test_agent Test [feature]
@system_test_agent Test [feature]
```

- Three-tier testing pyramid
- Each level validates before moving up
- Aligns with bottom-up development preference

Context Optimization

The template implements several strategies to maximize context efficiency:

Strategy	Implementation	Benefit
Progressive Disclosure	Detailed docs in <code>/docs</code> , loaded on-demand	Keeps root context minimal
Concise Root Context	<code>claude.md</code> under 60 lines	Initial context <20% of window
Just-in-Time Loading	Agents use <code>grep</code> / <code>find</code> before reading	Avoids unnecessary file loads
Context Isolation	Each agent has own 200k window	Preserves specialized knowledge
One Mission Per Session	Single focused task per agent	Prevents context contamination

MCP On-Demand	Tools loaded only when needed	Saves up to 95k tokens
---------------	-------------------------------	------------------------

Research Foundation

This template is built on insights from multiple authoritative sources:

Context Engineering

- **Source:** Anthropic Engineering Blog
- **Key Insight:** Progressive disclosure and just-in-time context loading optimize performance
- **Implementation:** `/docs` directory with on-demand file loading

Multi-Agent Orchestration

- **Source:** Azure AI Architecture Center
- **Key Insight:** Concurrent pattern best for parallelizable tasks; sequential for dependencies
- **Implementation:** Parallel planning phase, sequential implementation/review

Token Optimization

- **Source:** Reddit community research
- **Key Insight:** Initial context should never exceed 20% of window; MCP tools can consume 47%
- **Implementation:** Minimal `claude.md`, on-demand MCP loading

Context Isolation

- **Source:** Practical developer workflows
- **Key Insight:** Each specialist needs dedicated context window to preserve quality
- **Implementation:** Separate sub-agents with handoff artifacts

Performance Scaling

- **Source:** Anthropic multi-agent research system
- **Key Insight:** Token usage explains 80% of performance variance; multi-agent systems use $15\times$ more tokens but deliver 90% better results

- **Implementation:** Orchestrator-worker pattern with parallel execution

Customization

For Your Tech Stack

Edit `docs/architecture.md` to specify your:

- Programming languages
- Frameworks
- Databases
- Infrastructure

For Your Workflow

Modify agent definitions in `.claude/agents/` to match your:

- Development process
- Code review standards
- Testing requirements

For Your Team

Add new agents by:

1. Creating `.md` file in `.claude/agents/`
2. Defining role, responsibilities, workflow
3. Updating `claude.md` agent list

Best Practices

Parallel Execution

- Invoke multiple agents simultaneously for independent tasks
- Example: `@product_manager @ux_designer` for planning
- Each agent gets own context window

Sequential Handoffs

- Use handoff artifacts (like `task.md`) to coordinate
- Example: Planning → Implementation → Review

- Output of one agent becomes input of next

Context Management

- Use `/clear` and `/export` when approaching context limits
- Keep sessions focused on single mission
- Load documentation files only when needed

Quality Assurance

- Always run full review chain: Reviewer I → II → Cyber
- Execute testing pyramid: Unit → Integration → System
- Loop feedback to developers until all checks pass

Example: Building Authentication Feature

Bash

```
# 1. Plan (parallel)
@product_manager @ux_designer Create task.md for user authentication

# 2. Implement backend
@backend_developer Implement auth API from tasks/auth.md

# 3. Review backend (sequential)
@code_reviewer_I Review auth API
@code_reviewer_II Review auth API
@cyber_code_reviewer Review auth API

# 4. Test backend (sequential)
@unit_test_agent Test auth API
@integration_test_agent Test auth API

# 5. Implement frontend (can be parallel with backend if independent)
@ui_developer Implement login UI from tasks/auth.md

# 6. Review frontend (sequential)
@code_reviewer_I Review login UI
@code_reviewer_II Review login UI

# 7. Test frontend (sequential)
@unit_test_agent Test login UI
@integration_test_agent Test login flow
```

```
# 8. System test (final validation)
@system_test_agent Test complete auth flow

# 9. Document
@documentation_specialist Document authentication feature
```

Benefits

Speed

- **Parallel execution** reduces time for independent tasks
- **Dedicated context windows** prevent quality degradation
- **Automated workflows** eliminate manual handoffs

Quality

- **Multi-level review** catches issues at different abstraction levels
- **Bottom-up testing** ensures solid foundation
- **Specialized agents** provide expert-level analysis

Scalability

- **Reusable structure** works for projects of any size
- **Modular agents** can be added or modified independently
- **Clear workflows** make onboarding new team members easy

Maintainability

- **Comprehensive documentation** keeps knowledge accessible
- **Consistent conventions** reduce cognitive load
- **Living artifacts** evolve with the project

Requirements

- Claude Code CLI
- Git (for version control)
- Project-specific tools (defined in `docs/deployment.md`)

Resources

- **Usage Guide:** [USAGE_GUIDE.md](#)
- **Claude Code Docs:** <https://code.claude.com/docs>
- **Anthropic Best Practices:** <https://www.anthropic.com/engineering/claude-code-best-practices>
- **Context Engineering:** <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

License

This template is provided as-is for use in your projects. Customize freely to match your needs.

Contributing

To improve this template:

1. Test the workflow with your project
2. Document any optimizations or patterns you discover
3. Share feedback on what works and what doesn't

Built with insights from: Anthropic Engineering, Azure AI Architecture Center, and the Claude Code community.